实 验 报 告

姓名： 姚嫣如　　学号：21080012078
专业班级：23 网安　　课程：离散数学 II
上课老师：盛艳秀　　实验时间：2025/12/29

（一） 实验题目 ：无线基站频率分配问题

（二） 实验目的

1、掌握无向简单图（n ≤10，k≤4）的点色数求解方法；
2、掌握无向简单图色多项式的求解方法；
3、了解图的着色的实际应用。

（三） 实验要求

1、实验背景

在城市的无线通信网络规划中，为避免无线基站间信号干扰，需要给每个基站分配一个通信频段（频率）。由于**基站位置接近、信号范围重叠的基站不能使用相同频段**，这就构成了经典的**图点着色问题**：每个基站对应图中的一个顶点，若两个基站范围重叠则两个顶点间有一条边，**相邻顶点不能着同色**（即不能用同一频段）。

2、实验要求

为了提前评估频段资源的充足性，需要计算：
1、能否在 k 个频段（颜色）内完成频段分配（**合法着色**）？
2、给定 k 个频段，所有可能合法的频段分配**方案数**是多少？

（四） 实验内容和实验步骤；

一．需求分析

1、本实验旨在解决“无线基站频率分配问题”。通过将该实际问题抽象为图论中的“无向简单图点着色问题”，利用计算机算法判断在有限的频段资源（k 个颜色）下，是否能为所有基站（顶点）分配频段，使得相邻基站不产生信号干扰（相邻顶点颜色不同）。同时，计算出所有可能的合法分配方案总数，以评估频段资源的充足性。

2、输入形式

① 顶点数 n：一个整数，表示基站的数量。

② 频段数 k：一个整数，表示可用的频率数量。

③ 邻接矩阵 A：一个 n×n 的矩阵，其中：

- 若 A[i][j] 的值为 1，表示基站 i 与基站 j 范围重叠（有边相连）；
- 若值为 0，表示无重叠（无边相连）。
- 由于是无向简单图，矩阵应是对称的且主对角线全为 0。

3、输出形式

① 是否能完成频段分配：

输出“是/Yes”或“否/No”，明确回答在 k 个频段内是否存在合法分配方案

② 合法方案数：

输出一个整数，代表所有可能的合法频段分配（着色）方案的总数

4、数据约束

① 顶点规模：顶点数 n≤10（属于小规模图论问题）。

② 频段限制：频段数 k≤4。

③ 图的性质：输入必须为无向简单图（无环，无重边）且图是连通的。

5、程序所能实现的功能

① 合法性判断

根据输入的图结构和频段数 k，自动判定该图是否是一个 k-可着色图。

② 计算方案总数

计算并输出给定 k 个频段下，所有满足“相邻顶点不同色”条件的合法着色方案总数（即求**色多项式在 k 时的值 P(G,k)** ）。

二、概要设计

说明用到的数据结构定义、主程序的流程及各程序模块之间的调用关系。

1、数据结构定义

- 图的矩阵表示：

变量：A

类型：numpy.array

定义：一个 n×n 的二维整数数组。A[i][j] = 1 表示顶点 i 与顶点 j 之间有边（基站信号覆盖重叠），0 表示无边。

- 颜色状态数组：

变量名：color

类型：List[int] (列表)

定义：一个长度为 n 的列表，记录每个顶点的着色情况。color[i] 的值从 1 到 k 代表不同的颜色（频段），0 表示该顶点尚未着色。

- 计数器：

变量名：sum

类型：int

定义：用于累加并存储所有合法的着色方案总数。

2. 主程序流程

(1) 输入与初始化：

从控制台读取顶点数 n 和颜色数 k。获取用户输入的邻接矩阵，并验证输入格式。

(2) 递归求解阶段：

调用 count_schemes(0, ...)，从第 0 个顶点开始进行深度优先搜索。

(3) 结果输出阶段：

根据返回的方案总数，判断 k-着色的可行性，并输出具体的方案数值。

3. 程序模块之间的调用关系

main (主控模块)：作为调度中心，按顺序调用各功能模块。直接调用 count_schemes 启动搜索。

→ count_schemes() 函数;核心递归模块。它会循环尝试 k 种颜色，在每次尝试时调用 judeg_A 进行合法性检查。若检查通过，则向下递归调用自身 count_schemes(v + 1, ...)。

→ judeg_A() 函数：辅助模块。被 count_schemes 调用，通过遍历邻接矩阵的相关行，判定当前顶点与已着色的邻居顶点是否颜色冲突。

三、详细设计

```
import numpy as np

# 判断给顶点 v 涂上颜色 c 是否合法。
# v: 当前尝试涂色的顶点 color: 当前着色情况
def judeg_A(v, A, color, c):
    # 遍历，查找邻居
    for i in range(len(A)):
        if A[v][i] == 1 and color[i] == c:    # 邻居
```

```
                return False
        return True

# 为顶点 v 及其之后的顶点涂色，并返回合法方案总数。
# v：当前顶点 n：总顶点 k：可用颜色数
def count_schemes(v, n, k, A, color):
    if v == n:          #   0 到 n-1 的顶点都已经成功涂色了
        return 1

    count = 0
    # 为当前顶点 v 分配每一种可能的颜色
    for c in range(1, k + 1):
        # 检查是否与已涂色的邻居冲突
        if judeg_A(v, A, color, c):
            color[v] = c      # 如果不冲突，就把颜色 c 分配给顶点 v

            count += count_schemes(v + 1, n, k, A, color) # 给下一个顶点 (v + 1) 涂色

            color[v] = 0      # 回溯

    return count

# 主程序
def main():
    print("======== 无线基站频率分配（图点着色问题） ========")

    # 输入数据
    try:
        n = int(input("请输入顶点数 (n <= 10): "))
        k = int(input("请输入频段数 (k <= 4): "))

        print(f"请输入 {n}x{n} 的邻接矩阵: ")
        matrix = []
        for i in range(n):
```

```
            row_str = input(f"第 {i + 1} 行: ").split()  # 按照空格来分割字符
            row_int = [int(x) for x in row_str]  # 转换为 int
            # 输入判断
            if len(row_int) != n:
                print("该行输入错误!! 请重新输入! ")
                i -= 1
                continue
            matrix.append(row_int)

        # 使用 numpy 将列表转换为矩阵格式，方便处理
        A = np.array(matrix)

    except ValueError:
        print("输入格式有误，请输入整数。")
        return

    node_color = [0] * n            # 记录每个顶点的颜色

    # 从第 0 个顶点开始递归计算
    sum = count_schemes(0, n, k, A, node_color)

    # 4. 输出结果
    print("\n======== 实验结果 ========")
    if sum > 0:
        print(f"可以使用{k}种颜色完成频段分配")
    else:
        print(f"不能使用{k}种颜色完成频段分配")

    print(f"所有合法的频段分配方案数为：{sum}")

if __name__ == "__main__":
    main()
```

四、调试分析

1、调试过程中所遇到的问题及解决方法；

（1）问题：回溯状态残留。

- 现象：在计算方案总数时，如果某一条搜索路径结束后没有正确重置颜色，会影响其他路径的判定。
- 解决方法：在 count_schemes 递归调用返回后，立即执行 color[v] = 0。通过这一步“回溯”操作，确保了每一层搜索的独立性。

2、算法的时空分析；

（1）时间复杂度：

本算法采用回溯法（DFS）。在最坏情况下（如图是一个完全图或空图），对于 n 个顶点，每个顶点有 k 种颜色选择。

搜索树的节点数为 $O(k^n)$，每个节点进行冲突检查耗时 O(n)。

总时间复杂度为 $O(n \cdot k^n)$。由于实验约束 n≤10,k≤4，最大计算量约为 $10 \cdot 4^{10} \approx 10^7$，在现代计算机上可在 1 秒内完成。

（2）空间复杂度：

- 递归深度：递归深度为 n，因此递归栈空间开销为 O(n)。
- 存储空间：邻接矩阵占用 $O(n^2)$ 空间，颜色数组占用 O(n) 空间。
- 总空间复杂度为 O(n2)。对于 n=10 的规模，内存占用极低。

（五） 实验结果

列出对于给定的输入所产生的输出结果。若可能，测试随输入规模的增长所用算法的实际运行时间的变化。

1、$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$

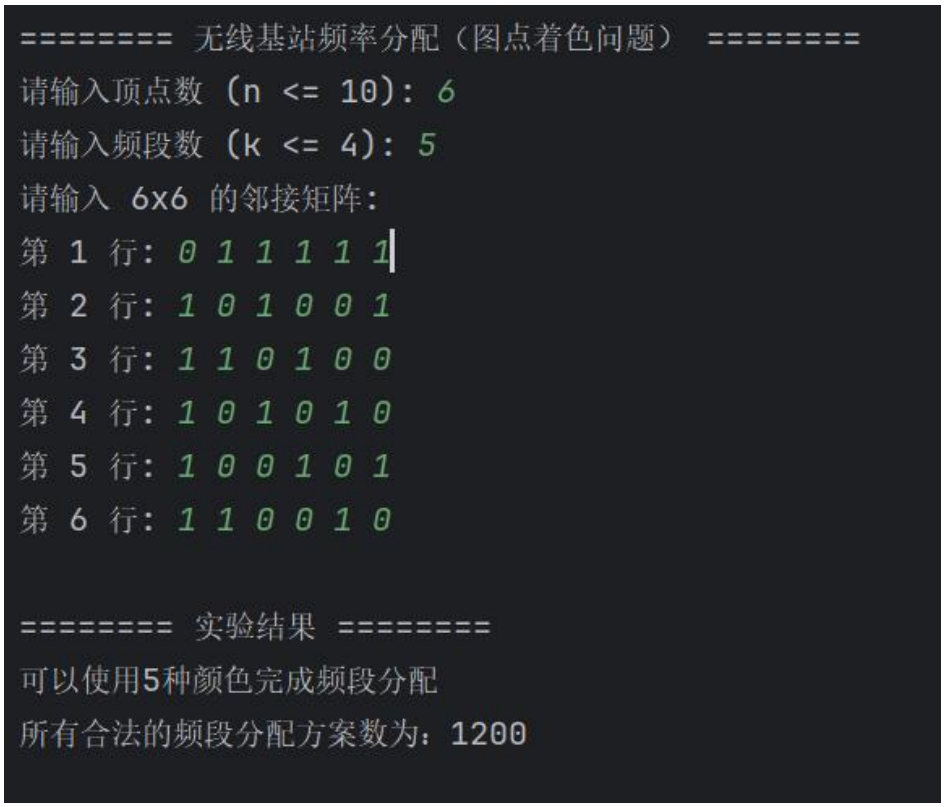
输出结果：

```
D:\编译器\python.exe D:\yyr\study\数学\离散2\lab4\离散
======== 无线基站频率分配（图点着色问题） ========
请输入顶点数 (n <= 10): 6
请输入频段数 (k <= 4): 3
请输入 6x6 的邻接矩阵:
第 1 行: 0 1 1 1 1 1
第 2 行: 1 0 1 0 0 1
第 3 行: 1 1 0 1 0 0
第 4 行: 1 0 1 0 1 0
第 5 行: 1 0 0 1 0 1
第 6 行: 1 1 0 0 1 0

======== 实验结果 ========
不能使用3种颜色完成频段分配
所有合法的频段分配方案数为：0
```

2、$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$

输出结果：



（六） 实验总结

通过本次实验，我深刻体会到了离散数学中图论知识在解决复杂工程规划问题中的核心作用。实验将“无线基站频率分配”这一实际需求抽象为经典的“无向简单图点着色问题”，将频段干扰的物理约束转化为数学上的顶点相邻约束。我理解了合法着色方案的数量不仅仅是一个组合数学的结果，它更直接反映了通信系统频段资源的充裕度与调度灵活性——合法的分配方案总数越多，意味着在实际部署中应对复杂电磁环境或特定基站硬件限制时的调整空间越大，系统的规划稳健性越强。

在算法实现过程中，通过对递归回溯法的编程，我深刻认识到图着色问题作为 NP-Hard 类问题的本质特征。虽然对于本实验中小规模的数据（$n \leq 10, k \leq 4$），$O(n \cdot k^n)$ 复杂度的搜索树尚可接受，但这让我意识到在处理大规模城市级通信网络时，简单的穷举和递归将面临“状态空间爆炸”的挑战。这种从代码执行效率到算法复杂度的直观感受，促使我思考在工程实践中如何权衡计算精度与效率，以及引入启发式算法或色多项式近似求解的必要性。

实验过程中的调试环节同样令我受益匪浅。通过对递归搜索树的逐层跟踪，我解决了回溯过程中“状态污染”的难题，验证了“重置颜色状态”对路径独立性的决定性作

用。从最初的逻辑报错到最终准确输出方案总数，这种从抽象矩阵到具体判定结果的闭环验证，极大地提升了我利用算法解决实际问题的信心。将数学理论转化为可运行的代码，不仅加深了对“色数”和“色多项式”概念的理解，更锻炼了我严谨的工程逻辑思维。